Workstation Migration

Bringing the Operator's Own Machine Into the Threat Model

Gurmann Basran

Prepared 2026-06-04. Revised 2026-06-15 (rev. 3).

1 What this document is

This is the story of the most recent milestone, which took the machine I administer the whole system from and rebuilt it to the same standard as everything it administers. I am giving it its own document because the reasoning is the interesting part, and because it is the clearest example in the project of a residual risk I named in writing and then went back to close. As with every other document here, the parts list is deliberately out: the distribution, the desktop, the terminal and shell environment, the editor, the bootloader, and the encryption layout are not named. What is in is the reasoning, the discipline, and an honest account of what the migration closes and what it leaves open.

2 The machine I had not thought about

For most of this project I hardened everything except the thing I was hardening it from. The infrastructure got a threat model, an audit, traceable findings, and two hardening passes. The workstation got none of that. It was a mainstream consumer operating system with a compatibility layer bolted on top to run the Linux tooling, and it was the single most-privileged endpoint in the environment: it held the keys, ran the administrative sessions, and could reach every other machine, while carrying the entire opaque attack surface of a closed consumer OS plus the seams of the shim stitched onto it.

The threat model document is honest about this. The "admin-workstation compromise" row in the residual-risk section has been there since the first version, and for a long time the only mitigations I could write in that row were partial ones. A fortress run from a soft machine is not a fortress; it is a soft machine with good intentions about the network behind it. This milestone exists because I got tired of the gap between how carefully I treated the servers and how casually I treated the laptop in front of them.

3 Bringing the threat model to the workstation

The goal was not a prettier desktop. The goal was to move the "admin-workstation compromise" row from mostly-unmitigated toward bounded, and to do it with the same discipline the rest of the project uses. Three changes carry most of the weight.

The first is that the workstation no longer holds the keys to the kingdom. Fleet access now rests on a short-lived signed certificate issued by a private certificate authority, not on long-lived per-host keys that sit on the disk indefinitely. A credential lifted off the machine ages out on its own, and a single central action revokes it for everything at once. The blast radius of a workstation

compromise is now the remaining lifetime of one short certificate, not the indefinite life of a pile of static keys.

The second is that the secrets were never on the machine to begin with. The actual credentials live in the vault, behind a second factor, and the workstation reaches them through gated access rather than caching them locally. Owning the machine does not hand an attacker the vault.

The third is the one this whole milestone is about: the machine itself was rebuilt to the same hardened standard as the infrastructure, on a native, minimal platform with full-disk encryption, rather than continuing to administer a fortress from a consumer box chosen for convenience. The most-privileged endpoint stopped being the least-examined one.

4 Non-destructive by design

I did not wipe the old environment, and that was a deliberate safety decision rather than a compromise. The new platform installs alongside the old one as a second boot entry; the old environment stays, reduced to exactly two jobs it is genuinely better at, gaming and one heavyweight development environment with no equivalent on the new platform, and is kept isolated from anything to do with fortress administration. Nothing that administers the fleet runs over there anymore.

The reason for keeping it rather than burning it down is the same reason every hardening phase in this project keeps a rollback path: the daily driver is itself production now, and a migration that leaves no way back is a migration that turns one bad afternoon into a week of recovery. The old environment is the rollback path. Booting it bare produces a games machine and an IDE, nothing more, and that is the whole point.

5 The configuration is the artifact

The entire workstation configuration is captured as a single reproducible kit that can rebuild the machine from a clean install deterministically. This is not a tidiness preference; it is the disaster-recovery story. A workstation you can only rebuild by remembering what you did to it over six months is a workstation you cannot actually rebuild. A workstation defined by a versioned, declarative configuration is one I can stand up again from bare metal in an afternoon, which means I can afford to treat the running machine as disposable.

That property also closes one of the lessons from earlier in the project. Configuration that lives only in a running machine's head, and never in version control, is exactly the kind of thing that drifts out from under your audits. Defining the workstation declaratively means the machine and its description are the same thing, and the description is in the same place I keep everything else I trust.

6 Reliability as a security property

A control you route around because it is annoying is a control you do not have. The same is true of a workstation: if the daily driver is flaky, I will quietly start doing administrative work from

somewhere less hardened to get my day done, and the careful posture rots from the inside. So the final piece of the milestone treats reliability as part of the security work, not separate from it.

The workstation now carries a manifest that declares each desktop component's expected role, its health check, its resource budget, and its rollback path. A validator reads the live state against that manifest. A constrained repair planner may take only pre-approved corrective actions. The user-session services are supervised so they come back when they fall over, and a pre-flight gate with a known-good visual baseline gives the machine a state to fall back to. The reasoning behind these is in the scripts document; what matters here is the intent. A daily driver earns the name only when it recovers from its own faults without me babysitting it, because a workstation I have to fight is a workstation I will eventually stop trusting, and stopping trusting it is how the soft-machine problem comes back.

The self-healing layer is now in place and verified, not just designed: it has caught and repaired real faults in normal use, and a known-good visual baseline plus a post-change verification pass gate every update so the machine always has a state to fall back to. Upgrades themselves go through a staged safety contract, where a rollback path is rehearsed before any change that could affect the boot, so an upgrade can never leave the machine unbootable with no way back. With the reliability layer proven, the migration off the old consumer environment is functionally complete; everything I administer the fleet with now runs natively, and the old environment is the games-and-one-IDE rollback path described earlier and nothing more.

What is being built on top is a terminal-native command center: a single keyboard-driven environment for running fleet operations from one place, with a pane per host, so the day-to-day of managing the system happens in one cockpit rather than a scatter of windows. Part of that is already usable, including a browser-reachable console that lets me reach the workstation from a locked-down machine without installing anything; it is gated by an identity check in front and a second authenticated layer behind it, and it is reached over an outbound-initiated path so nothing is exposed inbound. The rest is in progress rather than finished, and I would rather say so than imply a tidier ending than the one I have; the next hardening step on that console is a second factor, which is designed and queued rather than an open question, but it is not deployed yet.

7 What this closes, and what it does not

This is a hardened workstation, not an air-gapped one, and the difference is the residual risk. The machine still talks to the internet and still administers the fleet, so a browser-level or operating-system-level compromise of the workstation still owns the operator session and the short-lived certificate it currently holds, for as long as that certificate lives. The short time-to-live bounds the window; it does not eliminate it. Anyone who tells you a daily-use, internet-connected administrative workstation is fully defended against its own compromise is selling something.

What the milestone genuinely changed is the shape of that residual risk. Before, a compromise of the workstation meant long-lived keys to everything and secrets cached on the disk, with the machine itself an unexamined consumer OS. Now it means a single short certificate that expires and can be revoked centrally, secrets that were never on the disk, and a machine built and supervised to the same standard as the servers. The honest next step is a hardware second-factor that has to

be physically touched to authorize each privileged action, which would cut the residual window down to the moment of each action rather than the life of a certificate. That is future work and it is not deployed yet, so it sits in the residual-risk column of the threat model with a phase attached, which is the only place in this project a known gap is allowed to sit.

Two newer facts belong in this picture. The workstation now also holds the backup encryption key, kept immutable on disk with a verified off-fleet escrow copy, which makes this machine load-bearing for recovery as well as for administration. That is a deliberate concentration of trust, and the way it is made safe is the escrow rather than pretending the concentration is not there: the key the live backups are actually encrypted to has a current copy that lives off the machine and off the fleet, confirmed by deriving its identity and matching it, so the machine failing does not take the only copy of the key with it. The other fact is a deadline that turns all of this from theory into a test. A planned physical relocation of the hardware to a new site is coming, and a move is a forced full restore: it will exercise the recovery paths, the declarative rebuild of this machine, and the certificate re-issuance for real, under time pressure, away from the comfortable assumption that the running system will always just be there. That is the most honest validation a hardened workstation can get, and it is the reason recovery readiness is now part of the bar for this machine and not only for the servers behind it.

The portable lesson is the one I wish I had taken seriously a year earlier. The machine you operate from is part of the system you are defending, not a neutral window onto it. If you have a threat model, the workstation belongs inside it, and the day you write “admin-workstation compromise” into the residual-risk column is the day you have started owing yourself the work this document describes.